

Travaux dirigés n° 5

Graphes : plus courts chemins à origine unique

Exercice 1 (Préliminaires : initialisation et relaxation)

Les algorithmes de plus courts chemins qui font l'objet des exercices suivants sont basés sur la technique du relâchement. Pour chaque sommet $v \in S$, on gère un attribut $v.d$ qui est un majorant du poids d'un plus court chemin entre l'origine s et v . On appelle $v.d$ une estimation de plus court chemin. Au début de l'algorithme, on initialise les estimations à une valeur très grande, puis on les affine durant l'algorithme. De plus, on souhaite calculer non seulement les poids des plus courts chemins, mais aussi les sommets présents sur les plus courts chemins. Ainsi, pour chaque sommet $v \in S$, on gère un attribut $v.pere$ qui est le prédécesseur et qui est soit un autre sommet, soit NIL. Au début de l'algorithme, ces attributs sont initialisés à NIL pour chaque sommet. Ensuite, lorsque l'estimation d'un plus court chemin est affinée pour un sommet v , $v.pere$ est mis à jour selon la nouvelle estimation améliorante trouvée. À la fin de l'algorithme, en remontant la chaîne des prédécesseurs de v , on retrouve le plus court chemin entre s à v .

1°) Définissez une procédure **source-unique-initialisation** qui effectue l'initialisation des estimations et des prédécesseurs avant l'application d'un algorithme de plus courts chemins. Donnez son temps d'exécution.

Le processus de relâchement d'un arc (u, v) consiste à tester si l'on peut améliorer le plus court chemin vers v trouvé jusqu'ici en passant par u et, si tel est le cas, à actualiser $v.d$ et $v.pere$.

2°) Définissez une procédure **relacher** qui teste si le passage d'un sommet u vers un sommet v diminue la valeur de l'estimation de plus court chemin de v et qui met à jour son estimation et son prédécesseur en conséquence. Donnez son temps d'exécution.

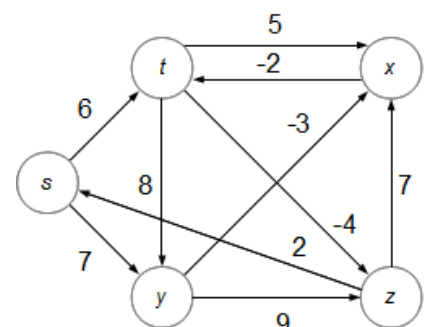
Exercice 2 (Algorithme de Bellman-Ford)

L'algorithme de Bellman-Ford résout le problème des plus courts chemins à origine unique dans le cas où les poids des arcs peuvent avoir des valeurs négatives. Étant donné un graphe orienté pondéré $G = (S, A)$ et l'origine s , il retourne une valeur booléenne indiquant s'il existe ou non un circuit de poids strictement négatif accessible à partir de l'origine. Si un tel circuit n'existe pas, l'algorithme donne les plus courts chemins ainsi que leurs poids. L'algorithme relâche les arcs, diminuant progressivement une estimation $v.d$ du poids d'un plus court chemin depuis l'origine s vers chaque sommet $V \in S$ jusqu'à atteindre le poids réel du plus court chemin.

Soit le graphe ci-contre :

1°) Illustrez l'exécution de l'algorithme de Bellman-Ford en démarrant du sommet s . Donnez le poids et l'arborescence des plus courts chemins obtenus.

2°) Définissez la procédure **pcc-bellman-ford** et donnez son temps d'exécution.



Exercice 3 (Le cas des graphes orientés acycliques)

En relâchant les arcs d'un graphe orienté acyclique $G = (S, A)$ dans l'ordre topologique des sommets, on peut calculer les plus courts chemins à partir d'une origine unique plus rapidement qu'avec l'algorithme de Bellman-Ford.

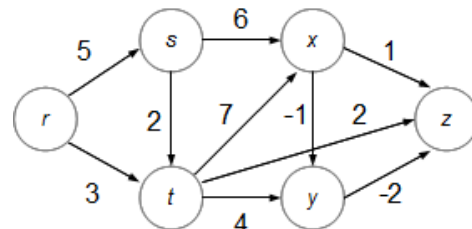
Soit le graphe orienté acyclique ci-contre :

1°) Triez topologiquement ce graphe.

Si le graphe contient un chemin allant du sommet u au sommet v , alors u précède v dans l'ordre topologique. Une seule passe doit alors être effectuée sur les sommets triés topologiquement. Pendant le traitement de chaque sommet, tous les arcs qui partent du sommet sont relâchés.

2°) Illustrez l'exécution de l'algorithme des plus courts chemins ainsi défini à partir de l'origine s . Donnez le poids et l'arborescence des plus courts chemins obtenus.

3°) Définissez la procédure `pcc-go` et donnez son temps d'exécution.



Exercice 4 (Algorithme de Dijkstra)

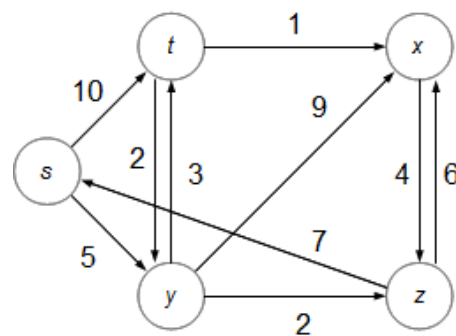
L'algorithme de Dijkstra résout le problème de la recherche d'un plus court chemin à origine unique pour un graphe orienté pondéré $G = (S, A)$ dans le cas où tous les arcs ont des poids positifs ou nuls. S'il est bien implémenté, cet algorithme a un temps d'exécution inférieur à celui de Bellman-Ford.

L'algorithme de Dijkstra gère un ensemble E de sommets dont les longueurs finales de plus court chemin à partir de l'origine s ont déjà été calculées. À chaque itération, l'algorithme choisit le sommet $u \in S - E$ dont l'estimation de plus court chemin est minimale, ajoute u à E , puis relâche tous les arcs partant de u . Afin de rendre efficace la sélection de ce sommet, on gère une file de priorités min F de sommets.

Soit le graphe ci-contre :

1°) Illustrez l'exécution de l'algorithme de Dijkstra en démarrant du sommet s . Donnez le poids et l'arborescence des plus courts chemins obtenus.

2°) Définissez la procédure `dijkstra` et donnez son temps d'exécution.



Exercice 5 (Plus courts chemins de tout sommet à tout sommet)

Définissez une procédure `pcc-tous-a-tous` qui calcule les plus courts chemins de tout sommet à tout sommet et donnez son temps d'exécution.

Références

Cormen T. H., Leiserson C. E., Rivest R. L. et Stein C., "Algorithmique" 3e édition, Dunod, 2010.